

Venus: Liquidity Hub

Security Review

Cantina Managed review by:
Valerian Callens, Lead Security Researcher
Matías Barrios, Security Researcher
MostafaYassin, Associate Security Researcher

August 11, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3
- 2.2 Trust assumptions . . . 3
- 2.3 Cantina Managed Team Statement . . . 4

3 Overview **5**
- 3.1 System overview . . . 5
- 3.2 Main components . . . 5
- 3.3 Integrated protocols . . . 5
- 3.4 Main flows supported . . . 5

4 Findings **6**
- 4.1 Medium Risk . . . 6
  - 4.1.1 FRV Accrued Yield Can Inflate Hub NAV Before It Is Liquid Or Certain . . . 6
  - 4.1.2 A residual in a zero-supply vault lets the performance fee charge a new depositor's principal, triggerable by a donation . . . 7
  - 4.1.3 Hub retained exit fee is bypassable via split redeem, undermining FRV settlement sandwich mitigation . . . 11
  - 4.1.4 Timestamp-based accrual guard skips Core vToken updates across same-timestamp blocks, enabling stale NAV share inflation . . . 14
- 4.2 Low Risk . . . 16
  - 4.2.1 Exit-fee HWM shield fails in drawdowns; performance fee skims retained exit-fee value from remaining LPs on recovery . . . 16
  - 4.2.2 Hub Share Pricing Can Use Stale FRV Lifecycle State . . . 18
  - 4.2.3 Emergency Reallocation Cannot Exit Locked FRV Positions . . . 18
  - 4.2.4 Redeemers can front-run an FRV settlement to exit at the pre-loss mark, socializing the shortfall to remaining holders . . . 19
  - 4.2.5 Selector collision can convert skippable external protocol reverts into hard withdraw failures . . . 20
  - 4.2.6 Urgent reallocation blocked when all destination caps are fully utilized . . . 20
  - 4.2.7 Collateral underdelivery compensation from Failed FRVs is outside Liquidity Hub token accounting and payout flow . . . 21
- 4.3 Informational . . . 22
  - 4.3.1 Resource Accrual Failures Are Silently Skipped . . . 22
  - 4.3.2 FRV Marked Value Can Consume YieldGroup Cap Room . . . 22
  - 4.3.3 Consolidated suggestions to clarify the documentation . . . 23
  - 4.3.4 A same-block deposit after unpause skips Core interest accrual, pricing shares on a stale rate and skimming pending interest . . . 24

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Venus Protocol is a leading lending and borrowing platform on the BNB Chain that prioritizes enhanced risk management, decentralization, and user experience.

From Jun 29th to Jul 9th the Cantina team conducted a review of venus-liquidity-hub on commit hash 5b9ebd4b (branch feat/VPD-1300). The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	4	2	2
Low Risk	7	4	3
Gas Optimizations	0	0	0
Informational	4	4	0
Total	**15**	**10**	**5**

2.1 Scope

The security review had the following components in scope for venus-liquidity-hub on commit hash 5b9ebd4b (branch feat/VPD-1300):

```
contracts/
├── Hub/lib/HubAdminLib.sol
├── Hub/Hub.sol
├── Hub/HubStorage.sol
├── YieldGroup/base/YieldGroupBase.sol
├── YieldGroup/base/YieldGroupBaseStorage.sol
├── YieldGroup/YieldGroup.sol
├── YieldGroup/YieldGroupFRV.sol
├── adapters/AdapterCoreV1.sol
├── adapters/AdapterFRV.sol
├── adapters/AdapterFlux.sol
├── interfaces/IHub.sol
├── interfaces/IMigrator.sol
├── interfaces/IResourceAdapter.sol
├── interfaces/IYieldGroup.sol
├── interfaces/IYieldGroupBase.sol
├── interfaces/IYieldGroupFRV.sol
└── migrator/Migrator.sol

deploy/
├── 01_DeployHubBeacon.s.sol
├── 02_DeployCore.s.sol
├── 03_DeployFRV.s.sol
├── 04_DeployFlux.s.sol
├── 05_DeployAssetVault.s.sol
├── 06_DeployMigrator.s.sol
└── DeployBase.s.sol
```

2.2 Trust assumptions

The review was conducted under the following trust assumptions:

- Governance and operational actors (VIP, Guardian, Operator) are assumed to perform their duties in accordance with documented procedures, mandates, and responsibilities. These actors are assumed to perform appropriate due diligence before authorizing on-chain actions. In particular, they will:

  - Ensure that addresses added as Resources or Adapters are not EOAs with an active EIP-7702 delegation;
  - Call `accrueFees()` before `setFeeRecipient(0)` if there is a deferred management fee remainder due to the cap;
  - Consider the size of the queues at both the Hub and Yield Group layers to avoid out-of-gas errors;
  - Batch consecutive operations into a single atomic transaction to prevent griefing attacks between these operations;
- Roles and permissions for privileged actions are assumed to be defined and managed by an `AccessControlledV8` contract. For the Hub, this contract is inherited directly. For Yield Groups, it is provided during initialization. The review assumes the following asymmetric permission model:
  - Governance can both relax and tighten protocol restrictions;
  - The Operator can only tighten protocol restrictions (e.g., pause operations, reduce caps, or reorder queues). It cannot relax them (e.g., unpause operations, increase caps, or register new components);
  - The Guardian can only pause the Hub;
- Ownership of newly deployed Hub proxies is assumed to be transferred to the Timelock contract, with ownership acceptance performed through the corresponding VIP.
- End users are considered untrusted and may behave arbitrarily or maliciously within the permissions granted by the system. Users are responsible for reviewing and understanding the on-chain actions they submit.
- The review assumes adherence to the principle of Segregation of Duties. No single actor is expected to simultaneously hold operational roles intended to remain independent, and collusion between incompatible roles is considered out of scope unless explicitly stated otherwise.
- The Liquidity Hub is assumed to support only standard ERC-20 tokens. Support for non-standard ERC-20 tokens (e.g., rebasing tokens, fee-on-transfer tokens, or tokens implementing transfer hooks) is outside the scope of this review.

2.3 Cantina Managed Team Statement

Venus engaged Cantina to conduct a security review of the Liquidity Hub. We would like to thank the Venus team for their responsiveness and constructive engagement throughout the review process. No significant issues were identified during the assessment, and the protocol is expected to operate as intended. The commits of the PR containing the fixes also includes two new, unrelated features (`HubRegistry.sol` and an optional consent hash). Changes can be seen in PR 4. As these fall outside the original audit scope, our review time was primarily focused on confirming that the fixes correctly remediated the issues identified during the audit. That said, the team reviewed the Registry and consent hash changes as well, and did not find anything concerning.

3 Overview

3.1 System overview

Venus Liquidity Hub is a per-asset allocator vault platform on BNB Chain. Each supported asset has one ERC-4626 Hub where users deposit an underlying token and receive Hub share tokens. Yield is generated by routing funds across multiple YieldGroups under governance policy, with Hub share value increasing via a rising exchange rate (no rebasing). The v1 will be deployed on BNB Chain with three yield families: Venus Core, Fluid Lending and Venus Fixed-rate Vaults (FRV).

3.2 Main components

- Hub (main entrypoint): handles deposits, withdrawals, share mint/burn, fee accounting, caps, queues, pauses, and reallocation orchestration.
- YieldGroups (Core / Flux / FRV): routing layers per protocol family, each with their own inner queue of resources.
- Adapters (Core, Flux, FRV): protocol-specific, stateless connectors that execute the underlying protocol calls through delegatecall.
- Resources: the actual third-party markets or vault positions (e.g., vToken/fToken/FRV vault shares).
- Migrator (optional user tool): helper contract to migrate a Venus Core position into the Hub.

3.3 Integrated protocols

- Venus Core (vTokens, with mint-redeemUnderlying model);
- Fluid Lending (fTokens, with ERC-4626 logic);
- Venus Fixed-Rate Vaults (FRV) (lifecycle-based fixed-rate vault products);

3.4 Main flows supported

- Deposit: User deposits into Hub → Hub routes assets through configured YieldGroups → YieldGroup routes through resources → adapter deposits into third-party protocol. If total routing capacity is insufficient, the transaction reverts (no partial fills).
- Withdraw/Redeem: User burns shares → Hub burns shares, uses idle assets first, then withdraws from YieldGroups/resources via queues → returns underlying. Reverts if liquidity is insufficient.
- Reallocation: Operator can rebalance TVL between YieldGroups and/or specific resources while keeping total assets net-zero. Emergency reallocation remains available when Hub is paused.
- Migration flow: Users can migrate a Venus Core position directly into the matching Hub.
- Governance/controls: multi-level pause and cap system, fee/EMA logic, and role-based actions for reconfiguration.

4 Findings

4.1 Medium Risk

4.1.1 FRV Accrued Yield Can Inflate Hub NAV Before It Is Liquid Or Certain

Severity: Medium Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `AdapterFRV.totalAssets()` values active FRV positions using principal plus accrued fixed-rate coupon. This value is included in `Hub.totalAssets()`, which is then used for Hub share pricing, previews, fee accrual, caps, deposits, mints, withdrawals, and redeems.

The issue is that this accrued FRV value may not be liquid or certain yet. During `Lock`, `PendingSettlement`, or `SettlementDeadlineExceeded`, the FRV position can be marked above principal even though users cannot necessarily withdraw that value from the FRV vault. If the Hub has other liquid assets, users may redeem Hub shares against those liquid assets at a share price inflated by the illiquid or unsettled FRV mark.

If the FRV later fails, liquidates, or settles below the marked value, remaining Hub shareholders absorb the loss while earlier redeemers may have exited at the optimistic price.

Proof of Concept: Assume Alice and an attacker both deposit into the Hub:

```
Alice deposits:    2,000 USDC
Attacker deposits: 2,000 USDC

Hub totalSupply:   4,000 shares
Initial share price: 1.00 USDC/share
```

The operator reallocates `1,000 USDC` into an FRV and leaves the rest liquid in another YieldGroup, such as Core:

```
Core liquid assets: 3,000 USDC
FRV principal:      1,000 USDC
```

The FRV enters `PendingSettlement`. The adapter marks the FRV at principal plus accrued fixed-rate coupon:

```
FRV marked value: 1,072 USDC
Hub totalAssets:  3,000 + 1,072 = 4,072 USDC
Hub share price:  4,072 / 4,000 = 1.018 USDC/share
```

The attacker expects the FRV to settle below the marked value. Before settlement is finalized, the attacker redeems all `2,000` Hub shares:

```
Attacker receives: 2,000 * 1.018 = 2,036 USDC
```

The withdrawal is paid from liquid Core assets, not from the locked FRV:

```
Core liquid assets after attacker exit: 3,000 - 2,036 = 964 USDC
FRV still marked:                       1,072 USDC
Remaining Hub shares:                   2,000
```

The FRV then settles badly, for example at only `600 USDC`:

```
Real Hub assets: 964 + 600 = 1,564 USDC
Alice shares:    2,000
Alice receives:  1,564 USDC
```

Final result:

```
Attacker deposited: 2,000 USDC
Attacker withdrew:  2,036 USDC
Attacker profit:    36 USDC

Alice deposited:    2,000 USDC
Alice withdrew:     1,564 USDC
Alice loss:         436 USDC
```

The exploiter exits before the FRV is repriced down. The withdrawal is possible because the Hub has liquid sibling assets, while the share price is inflated by the unsettled FRV mark. Once the FRV settles below the mark, the remaining shareholder absorbs the shortfall.

An FRV vault can settle below the expected price mainly when a borrower defaults, enters bad debt, or gets liquidated.

Recommendation: This is an issue that arises due to the design of the system. So the fix would require re-thinking the design decisions. One possible fix is to avoid accounting for the interest for the FRV and only account for the principal amount, because this is what is available.

However, this leaves the issue of the FRV liquidity not being available until it matures and settles. So FRV liquidity cannot be used for withdrawal/redemption as the other resources.

Venus: This is the known gap. Please have a look at PR 2 comment thread you'll get the full context from the discussion. If you have a better recommendation or an alternative approach, please let us know.

Cantina Managed: Acknowledged.

4.1.2 A residual in a zero-supply vault lets the performance fee charge a new depositor's principal, triggerable by a donation

Severity: Medium Risk.

Context: *(No context files were provided by the reviewer)*.

Description: The `Hub` charges a performance fee on the gain of price-per-share above a monotonic high-water mark (HWM), computed in `_pendingPerformanceFee`. The HWM is seeded at launch to `EXP_SCALE / 10^decimalsOffset` and advances only on a new high in `_accrueFees` or by the retained exit-fee uplift in `_withdraw`.

Whenever the vault holds `totalSupply() == 0` with `totalAssets() > 0`, the HWM is left stale, because neither path that updates it runs at zero supply. `_accrueFees` early-returns on `ts == 0` (`Hub.sol:998-1002`), so it never resets the HWM:

```
uint256 ts = totalSupply();
if (ts == 0) {
    _lastFeeAccrualTimestamp = nowTs;
    return;
}
```

And the exit-fee HWM advance in `_withdraw` is gated on `ts != 0` (`Hub.sol:919-929`), which is false post-burn on the last exit:

```
if (_redeemFeeBps != 0) {
    uint256 ts = totalSupply();
    if (ts != 0) {
        uint256 newPps = (totalAssets() * EXP_SCALE) / ts;
        uint256 hwm = _highWaterMarkPerShare;
        if (newPps > hwm) {
            _highWaterMarkPerShare = newPps;
            emit HighWaterMarkUpdated(hwm, newPps);
        }
```

```
    }
}
```

Because the current code never re-anchors the HWM when supply restarts from zero, the next deposit is priced against the HWM the empty vault still carries, and its performance fee is computed wrong, harming either the depositor or the protocol. Two factors decide the outcome:

- The carried HWM is either at its default launch value, on a freshly initialized vault or one emptied after a cycle with no yield, or raised above it by a prior cycle's yield.
- The empty vault either holds a residual, from a retained exit fee or a donation, or is clean.

Empty vault	HWM at its default	HWM raised by prior yield
Holds a residual	Over-charge	Over-charge
No residual	Neutral	Under-charge

The combinations collapse into two harmful impacts, explained below. A clean deposit onto a vault still at its default HWM is the neutral, expected case and not a defect.

Over-charge: the depositor loses: A residual left in the empty vault makes the first deposit price above the old HWM. It comes from one of two sources:

- Exit fees retained in the vault when the last liquidity provider claims out and leaves it empty.
- A malicious actor donating the asset right after the vault becomes empty.

Either way, the performance fee reads the gap as a gain and mints it on the depositor's principal, though no yield was earned, so the depositor and the holders present lose that value to the fee recipient. This happens whether or not the old HWM was raised by a prior cycle, since the residual pushes the entry price far above it either way.

The donation source is the sharpest, and it does not even need a prior liquidity provider: on the first deposit into a freshly deployed vault, a malicious actor can donate a tiny amount, even a single token, to create the residual and trigger the mispricing. The harm peaks for a small donation, so this costs the attacker almost nothing while making the next depositor lose close to the full performance rate of their deposit.

Example, with `performanceFeeBps = 2000`:

- The vault is empty: 0 shares, no yield, and the HWM at its launch value of 1.0.
- An attacker donates 1 token, so the vault holds 1 token but still 0 shares.
- Bob deposits 1000 tokens; priced against the 1-token residual on zero prior shares, his entry price-per-share lands far above the launch HWM of 1.0.
- Since the price sits far above the HWM, the fee treats almost the entire 1001-token vault as a gain and mints 20% of it, about 200 tokens of shares, to the fee recipient.
- Bob's position is now worth about 800 tokens, a loss of about 20% of his 1000-token deposit, for a 1-token attacker cost.

The same over-charge occurs with no donation when the residual is a retained exit fee instead: if the last liquidity provider exits leaving, say, 50 tokens of fee in the vault, the next depositor is over-charged the same way, only the residual arises on its own rather than from an attacker.

Under-charge: the protocol loses: When the vault empties cleanly after a profitable cycle, the new deposit enters at the launch price, below the old HWM the prior cycle raised. No performance fee is charged until the price climbs back above that old HWM, so the new cycle's yield up to the old high is never collected and the protocol loses that revenue. This holds whether the old HWM sits a little or far above the new entry.

Example, with `performanceFeeBps = 2000`:

- Alice deposits 1000 tokens into a fresh vault; the price-per-share starts at 1.0.

- Her position earns 1000 tokens of yield, so the vault holds 2000 and the price-per-share doubles to 2.0.
- The HWM ratchets to 2.0.
- Alice fully exits, leaving the vault clean and empty.
- The HWM stays at 2.0, never reset.
- Bob deposits 1000 tokens, entering at a price-per-share of 1.0, half the old HWM.
- Bob's position earns 500 tokens of yield, lifting the price-per-share to 1.5.
- 1.5 is still below the HWM of 2.0, so no performance fee is charged.
- The fee recipient forgoes about 100 tokens, the 20% it would have collected had the HWM been reset to Bob's entry of 1.0.

Note: the size of the loss depends on how the first deposit's price compares to the old HWM; the further the old HWM sits above the new entry, the more of the new cycle's yield goes uncharged.

Proof of Concept: Both paths reproduce with `decimalsOffset = 6`, an 18-decimal asset, and `performanceFeeBps = 2000`, with no yield earned.

Supply-reset path (`redeemFeeBps = 500`):

```
// Actors are pre-funded and have approved the Hub.
// Alice deposits and fully exits as the last LP; the 5% exit fee is retained.
vm.prank(alice);
hub.deposit(1000e18, alice);
vm.prank(alice);
hub.redeem(hub.balanceOf(alice), alice, alice);

assertEq(hub.totalSupply(), 0);               // supply reset to zero
assertEq(hub.totalAssets(), 50e18);           // residual: the retained exit fee
// nothing resets the HWM while ts == 0, so it stays at its stale launch value

// Bob deposits into the reset vault, then anyone accrues fees next block (no yield).
vm.prank(bob);
hub.deposit(1000e18, bob);
vm.warp(block.timestamp + 1);
hub.accrueFees();

// feeRecipient was minted ~202e18 of shares on principal (about 20% of TVL);
// Bob's position is now ~808e18, a ~19% loss on his 1000e18 deposit.
assertLt(hub.convertToAssets(hub.balanceOf(bob)), 900e18);
```

Donation-griefing path (no exit fee), exploitable by a one-token donation:

```
// The donation is deliberately small next to Bob's 1000-token deposit, one token here.
// The loss peaks for a small donation and backfires for a large one, so this is the
↪  worst case.
vm.prank(attacker);
asset.transfer(address(hub), 1e18);           // counted as Hub idle

assertEq(hub.totalSupply(), 0);
assertEq(hub.totalAssets(), 1e18);            // residual donation

// Bob is the first depositor, then anyone accrues fees next block (no yield).
vm.prank(bob);
hub.deposit(1000e18, bob);
vm.warp(block.timestamp + 1);
hub.accrueFees();

// feeRecipient was minted ~200e18 on principal for a 1-token attacker cost;
// Bob's position is now ~800e18, a ~20% loss on his 1000e18 deposit.
assertLt(hub.convertToAssets(hub.balanceOf(bob)), 900e18);
```

Recommendation: Reset the HWM to the entry price-per-share whenever total supply restarts from zero, on the first-ever deposit or after a full exit. The correct fix is to add the following reset to `_deposit`:

`contracts/Hub/Hub.sol`

```
  function _deposit(address caller, address receiver, uint256 assets, uint256 shares)
  ↪  internal override {
+     bool wasZeroSupply = totalSupply() == 0; // first-ever deposit or a post-exit
↪  restart
      IERC20Upgradeable(asset()).safeTransferFrom(caller, address(this), assets);
      _routeDeposit(assets);
      _mint(receiver, shares);
+     // Reset the HWM to the entry price, up or down (the post-mint supply check guards
↪  the /0).
+     if (wasZeroSupply && totalSupply() != 0) {
+         _highWaterMarkPerShare = (totalAssets() * EXP_SCALE) / totalSupply();
+     }
      emit Deposit(caller, receiver, assets, shares);
  }
```

The HWM is a per-share baseline, and the performance fee is charged on the gap between the current price and it:

```
// _pendingPerformanceFee: the fee base is the rise of the price above the HWM
uint256 profitAssets = ((currentPps - hwm) * ts) / EXP_SCALE;
```

Across a supply reset the share scale changes and the depositor cohort is entirely new, so a HWM carried from the previous cycle no longer describes the new one and can be wrong in either direction:

- Carried HWM too low, over-charge. A residual left in the empty vault, from a retained exit fee or a malicious donation, inflates the new entry price far above the stale HWM, so `currentPps - hwm` reads as a large gain and the fee is minted against principal. This is the finding above.
- Carried HWM too high, under-charge. After a profitable cycle the HWM has ratcheted up; a later clean restart enters at the launch price, below that stale-high HWM, so the new cycle's real yield up to the old high goes untaxed until `currentPps` climbs back above `hwm`.

Anchoring the HWM to the actual entry price on every restart is always the right baseline, because each restart begins a fresh cohort whose reference is their own entry, not the HWM left by depositors who have already exited. It makes `currentPps - hwm` start at zero: a residual is no longer read as a gain, and a stale high no longer sits above the entry to suppress fees. Resetting up or down, rather than only raising the HWM, is what covers both directions.

This is safe against fee evasion, since a full exit runs `_accrueFees` before the burn and realises the exiting cycle's performance fee while supply is still nonzero, so resetting the next cycle's baseline forfeits nothing.

The launch-time seed in `initialize` sets the HWM to the launch price-per-share:

```
// initialize
_highWaterMarkPerShare = EXP_SCALE / (10 ** uint256(decimalsOffset_));
```

This value is fixed at deploy because the `decimalsOffset` pins the first-deposit exchange rate: a clean first deposit of any amount `x` mints `x * 10^decimalsOffset` shares, so the resulting price `x * EXP_SCALE / (x * 10^decimalsOffset)` cancels `x` down to `EXP_SCALE / 10^decimalsOffset`. The deposit size does not affect it, which is why it can be seeded before anyone deposits.

This reset leaves the launch-time seed in place; the seed is safe to keep and complementary to it, so `initialize` need not change. The two are consistent: on a clean first deposit the reset recomputes exactly `EXP_SCALE / 10^decimalsOffset` and matches the seed, and it only moves the HWM when a residual makes the real entry price diverge, which is the case the seed alone cannot capture. Keeping the seed is also independently safe, since the HWM is read only when supply is nonzero, `_accrueFees` returns early at `ts == 0` and the `_withdraw` branch is gated on `ts != 0`, and by then the first deposit has already set it. The seed therefore stays as a consistent fallback rather than a competing source of truth.

Venus: Fixed in commit 9ae379a.

Cantina Managed: Fixed in the commit above. `_deposit` now re-anchors the HWM to the new cohort's entry price-per-share on a refill from empty, up or down, so a residual no longer reads as a gain against the depositor's principal and a stale-high HWM from a prior cycle no longer suppresses the new cycle's fee:

```
uint256 preTs = totalSupply();
//...
_mint(receiver, shares);
if (preTs == 0) {
    uint256 ts = totalSupply();
    if (ts != 0) {
        uint256 entryPps = (totalAssets() * EXP_SCALE) / (ts + 10 **
        ↪  uint256(_decimalsOffset()));
        uint256 seedPps = EXP_SCALE / (10 ** uint256(_decimalsOffset()));
        if (entryPps < seedPps) entryPps = seedPps;
        uint256 hwm = _highWaterMarkPerShare;
        if (entryPps != hwm) {
            _highWaterMarkPerShare = entryPps;
            emit HighWaterMarkUpdated(hwm, entryPps);
        }
    }
}
```

The anchor is hardened against dust-refill manipulation in both directions. Pricing it off the virtual-share-inclusive supply (`ts + 10^offset`) holds it at or below the honest entry price, so a dust refill cannot inflate the HWM to disable the performance fee; and flooring it at the launch seed keeps a dust refill from pinning it low to tax the next depositor's principal. The HWM lands at or slightly below the raw entry price, so the depositor's principal is protected while at most a bounded slice of the residual windfall, about the performance rate of that residual, is charged. The `_withdraw` exit-fee shield and the `_accrueFees` ratchet are untouched, and the refill path is the only one that can lower the HWM, gated on the supply zero-to-nonzero transition.

4.1.3 Hub retained exit fee is bypassable via split redeem, undermining FRV settlement sandwich mitigation

Severity: Medium Risk.

Context: *(No context files were provided by the reviewer)*.

Description: The Hub's exit fee `redeemFeeBps` is the anti-sandwich friction meant to discourage a redeemer from racing out ahead of a downward FRV settlement reprice. An FRV position is marked at principal plus accrued coupon before settlement and can step down when the loan settles short, so the fee is intended to make exiting just before that reprice unprofitable. The fee defaults to 0 and is armed by governance through `setRedeemFeeBps`, so the behavior below applies once governance has set it.

Unlike a treasury fee, the exit fee is retained in the vault. On a redeem the Hub burns more share-value than the assets routed out, which lifts price-per-share for the remaining holders, and `Hub._withdraw` advances the high-water mark to match that uplift:

```
// Hub._withdraw
if (_redeemFeeBps != 0) {
    uint256 ts = totalSupply();
    if (ts != 0) {
        uint256 newPps = (totalAssets() * EXP_SCALE) / ts;
        uint256 hwm = _highWaterMarkPerShare;
        if (newPps > hwm) {
            _highWaterMarkPerShare = newPps;
            emit HighWaterMarkUpdated(hwm, newPps);
        }
    }
}
```

`previewRedeem()` and `previewWithdraw()` apply `redeemFeeBps` to the payout, so the fee never leaves the pool.

Because the fee is retained and lifts price-per-share, and ERC-4626 allows a holder to call `redeem()` or `withdraw()` repeatedly, a redeemer who still holds shares recaptures part of their own fee through the price-per-share uplift on those remaining shares. Splitting one exit into several sequential redeems therefore drives the effective fee well below the nominal `redeemFeeBps`. Two structural weaknesses in the fee as a deterrent follow:

1. **Recapturable by splitting.** On a single-shot exit the redeemer is gone, so the whole retained fee benefits the other holders. On a gradual exit the redeemer captures their shrinking share of each retained fee, so the more they split, the less net fee they pay.
2. **Hub-wide, not targetable.** `redeemFeeBps` is a single Hub-level parameter, so raising it ahead of one risky FRV settlement taxes every redemption from the Hub across all holders and all YieldGroups, not just the run it is meant to deter.

The recaptured fraction is bounded by how much of the pool the redeemer holds, not by the split count, and only a handful of redeems reach that floor. A small holder merely shaves the fee, while a holder of nearly the entire pool drives the effective fee to about zero and exits with no penalty, since almost all of each retained fee returns to their own remaining shares. The meaningful case is therefore a large or dominant holder, who can also enlarge their fraction by depositing before the run.

Risk:

When governance arms `redeemFeeBps` to deter a run ahead of a risky FRV settlement, a redeemer can defeat the intended friction by splitting the exit into several sequential `redeem()` or `withdraw()` calls and recapturing a growing fraction of the fee through the price-per-share uplift on their remaining shares. The effective fee drops below the configured `redeemFeeBps`, so a settlement run that a single-shot fee would leave unprofitable can turn profitable. The value the runner extracts by exiting at the pre-reprice mark is then socialized to the remaining holders when the FRV position settles down.

Because `redeemFeeBps` is a single Hub-level parameter, arming it also taxes every unrelated redemption across all YieldGroups, so the friction cannot be aimed at the specific settlement it is meant to guard.

Proof of Concept: A Foundry test on the Hub fixture reproduces the recapture with two equal holders and a 5% exit fee.

1. Two holders, alice and bob, each deposit 1000e18 into a Hub wired to a single YieldGroup, and governance arms `redeemFeeBps = 500`.
2. `previewRedeem(aliceShares)` gives the exact payout of a single-shot exit with no state change, so it can be compared against a real split from the same state.
3. Alice instead redeems her shares in 16 equal chunks; each retained fee lifts price-per-share and her still-held shares capture part of it.
4. The net assets alice receives across the 16 redeems are summed and compared against the single-shot payout.

```
uint256 aliceShares = hub.balanceOf(alice);
uint256 grossValue = hub.convertToAssets(aliceShares);
uint256 netOneShot = hub.previewRedeem(aliceShares); // single-shot payout, no state
↪  change

uint256 n = 16;
uint256 chunk = aliceShares / n;
uint256 netSplit;
for (uint256 i = 0; i < n - 1; ++i) {
    netSplit += _redeemAs(alice, chunk);
}
netSplit += _redeemAs(alice, hub.balanceOf(alice)); // remainder

assertGt(netSplit, netOneShot, "splitting recaptures part of the retained exit fee");
```

Result: the single-shot exit nets `950e18`, an effective fee of 500 bps as configured. The 16-way split nets `963.96e18`, recapturing `13.96e18` and dropping the effective fee to 360 bps.

The recapture saturates quickly in the number of splits, so only a handful are needed: for the same 50% holder the effective fee is 368 bps at 8 splits, 360 bps at 16, and 353 bps at 128, so a few redeems capture almost all of it and further splits add only a few bps.

At that same 16 splits, the recapture depends on the redeemer's pool fraction. Against the armed fee of 500 bps:

1. 1% of the pool: effective fee 497 bps.
2. 20% of the pool: effective fee 451 bps.
3. 50% of the pool: effective fee 360 bps.
4. 90% of the pool: effective fee 151 bps.
5. 99% of the pool: effective fee 53 bps.
6. 99.9% of the pool: effective fee 38 bps.

The recapture is negligible for a small or average holder, who is charged essentially the full fee: a 1% holder still pays about 497 bps and a 20% holder about 451 bps. It only becomes a large bypass for a holder that owns most of the pool, where a 99.9% holder pays about 38 bps, and lower still with more splits. The meaningful case is therefore a dominant holder, whose exit the fee no longer meaningfully deters.

Recommendation: Consider that the recapture cannot be removed from the retained fee directly: retaining the fee in the vault is exactly what socializes it to the holders who stay, and the same price-per-share uplift that socializes it is what the redeemer's own remaining shares recapture. A single price-per-share cannot leave value for the holders who stay without also crediting the exiting holder's remaining shares, and a per-account restriction does not help since the exit can be split across several addresses. The recapture only reaches a full bypass for a holder that owns most of the pool, so the exposure scales with how concentrated a single account can become in one Hub. Neither option below is free.

Escrow the exit fee and release it to the remaining holders at settlement: Instead of retaining the fee in the vault, hold it in a separate escrow that does not lift price-per-share on exit, and pay it to the holders who remain when the FRV settles. Each redeem then becomes a clean proportional exit with no retained uplift to recapture.

- Pro: removes the recapture, since the fee no longer lifts price-per-share on exit. The exit-side change is self-contained to where `previewRedeem` and `_withdraw` direct the fee.
- Pro: keeps the fee's socialization, since the holders who stay receive it at settlement.
- Pro: regains the fee's deterrent function, since a runner who has exited before settlement cannot recapture it, so the fee actually costs the run it was meant to deter.
- Con: releasing the escrow into `totalAssets()` at settlement is a discrete price-per-share step, a small version of the jump the linear accrual smooths. Its size is only the escrowed exit fees, far below the full coupon that the linear accrual keeps a late depositor from front-running, so the sandwich it exposes is a small fraction of the original one; a gradual accumulator would remove even that at the cost of more machinery.
- Con: it only helps when the FRV settles below its accrued mark, a shortfall the Venus team treats as a low-probability event for its vetted institutional counterparties, so the protection it adds may be small relative to the disruption and machinery it introduces.

Keep the retained fee and set its value with this limitation in mind: If the fee is kept as it is, treat it as a soft, position-dependent friction rather than a hard per-settlement deterrent: it deters a small or average holder but is progressively recaptured by a large one and nearly fully bypassed by a holder that owns most of the pool, while the downward FRV risk stays bounded by the exposure cap and treasury backstop. The Venus team may reasonably decide this is acceptable and not worth changing the implementation, but the value chosen for `redeemFeeBps` should be set with this tradeoff in mind, rather than on the assumption that the nominal fee is what a runner pays.

Venus: Agree, we confirm the behavior: because the exit fee is retained in the vault and socialised to remaining holders as a price-per-share uplift, a holder who still holds shares recovers a portion of the fee on those remaining shares, so splitting an exit into several sequential redemptions lowers the effective fee. As you note, the recaptured fraction scales with the holder's share of the pool and saturates after a few splits, so a small holder pays close to the nominal rate while a holder of most of the pool recaptures the majority of it. We are keeping the retained-fee design (your second option). The socialisation to remaining holders is intentional: the fee is meant to compensate the LPs who stay through a downward settlement, and retaining it in the vault is what delivers that. We will treat `redeemFeeBps` as a soft, position-dependent friction rather than a hard per-settlement deterrent, and we will choose its value with this recapture tradeoff in mind rather than assuming a redeemer pays the full nominal rate.

We are not adopting the escrow approach. Releasing the escrowed fee into `totalAssets()` at settlement is itself a discrete price-per-share step, which reintroduces a smaller version of the jump the FRV linear accrual is designed to smooth, and the machinery it requires is disproportionate to a below-mark settlement, which we treat as a low-probability event for our vetted counterparties. Alongside the exit fee, exposure to any single FRV position is bounded by its per-YieldGroup cap, and governance retains pause controls for a settlement known to be at risk.

We consider this finding acknowledged and are accepting the current implementation with the fee value set accordingly.

Cantina Managed: Acknowledged.

4.1.4 Timestamp-based accrual guard skips Core vToken updates across same-timestamp blocks, enabling stale NAV share inflation

Severity: Medium Risk.

Context: *(No context files were provided by the reviewer)*.

Description: The Hub relies on a timestamp-equality guard in `_accrueFees()` to enforce per-block idempotence when updating fee and resource state:

```
function _accrueFees() internal {
    uint64 nowTs = uint64(block.timestamp);
    uint64 last = _lastFeeAccrualTimestamp;
    if (nowTs == last) return;

    // Settle resource interest before reading NAV so the fee is charged on
    ↪  interest-current
    // value, not a one cycle stale one. The timestamp guard above bounds this to once
    ↪  per block.
    _accrueResources();
```

On chains such as BNB Chain (with a block time of `0.45s`), however, consecutive blocks frequently share the same timestamp. For instance, these three consecutive blocks share the same block timestamp `1783098780`:

- Block 107866664.
- Block 107866665.
- Block 107866666.

This causes the Hub to incorrectly skip `_accrueResources()` across distinct blocks, even though Core resources (`vToken`) accrue interest on a per-block basis:

```
// https://github.com/VenusProtocol/venus-protocol/blob/5d97ae974fed088bcf3a9dab325fce0e
↪  c1d6a493/contracts/Tokens/VTokens/VToken.sol#L571

function accrueInterest() public virtual override returns (uint) {
    /* Remember the initial block number */
    uint currentBlockNumber = block.number;
    uint accrualBlockNumberPrior = accrualBlockNumber;
```

```
/* Short-circuit accumulating 0 interest */
if (accrualBlockNumberPrior == currentBlockNumber) {
    return uint(Error.NO_ERROR);
}
```

Hub share pricing occurs immediately after `_accrueFees()` and before routing deposits to adapters. The system prices new Hub shares using stale Core `vToken` NAV obtained by calling `totalAssets()` (under the hood, `AdapterCoreV1.totalAssets()` fetches `vToken.exchangeRateStored()`), while vToken interest for the new block is only realized later during the execution of `vToken.mint()`. This results in inflated Hub share calculation and issuance, and subsequent value extraction when the attacker redeems at fresh NAV. It actually affects all main actions offered by the Hub (deposit, mint, withdraw, redeem) since they all call `_accrueFees()` before `ERC-4626` computes shares from `totalAssets()`, so stale valuation directly affects share mint/burn amounts.

The vulnerability is permissionless and repeatable on each same-timestamp block, which happens for approximately `55%` of blocks on BNB chain with a block time of `0.45s`, and directly transfers the missed block-level interest from existing LPs to the attacker, not the principal. For each attempt, the extractable amount per event is bounded by the skipped per-block Core accrual, and may become meaningful depending on how large is the attacker capital relative to TVL. Otherwise gain is small (linear in capital share).

The root cause of that issue is an incorrect assumption that timestamp equality is a reliable per-block discriminator. Core valuation of vTokens is block-lazy and requires per-block accrual.

Proof of Concept: The following simplified scenario illustrates the differences between using block-based or timestamp-based accrual in the case of a hub deposit executed on block `N+1`, when block `N+1` has the same timestamp as block `N`, and a Hub accrual happened at block `N`.

Scenario:

- Initial state:
  - Block `N` (number: `55`, timestamp: `55555`).
  - Honest LP already deposited `2000`.
  - A hub action at block `N` triggered an accrual at block `N` at Hub and Core vToken levels:
    - `hub._lastFeeAccrualTimestamp == 55555`
    - `vToken.accrualBlockNumber == 55`
  - `TVL = 2000`
  - `TotalShares = 1000`
  - Block accrual rate = `+100` per block (Core vToken lazy, block-based). Value is intentionally high to illustrate what happens.
- Attacker deposits `1000` at block `N+1`, with same timestamp (number: `56`, timestamp: `55555`).
- A block-timestamp collision happens, and results in extracted value for the attacker.

Steps	Block-based accrual (Hub stores the last accrual in `lastBlock`)	Timestamp-based accrual (Hub stores the last accrual in `lastTs`)
1) State before attacker action, at block `N` (number: `55`, timestamp: `55555`), where a hub action accrued	`lastBlock = 55`, `TVL = 1000`, `TotalShares = 1000`	`lastTs = 55555`, `TVL = 1000`, `TotalShares = 1000`
2) Attacker executes a deposit at block `N+1`	Attacker transaction lands in block `56`, timestamp `55555`	Attacker transaction lands in block `56`, timestamp `55555`

3) `_accrueFees()` guard	`block.number 56 != lastBlock 55` → accrual runs, `accrueResources()` executes	`nowTs 55555 == lastTs 55555` → early return, `accrueResources()` skipped
4) NAV before mint	`vToken.exchangeRateStored` was accrued, so it reflects interests for block `56` : `TVL = 2100` (includes `+100`)	`vToken.exchangeRateStored` was not accrued, so it does not reflect interests for block `56` : `TVL = 2000` (misses same-block yield update)
5) Shares to mint are calculated for the attacker	`shares = 1000 * 1000 / 2100 = 476`	`shares = 1000 * 1000 / 2000 = 500`
6) Post-deposit Hub state	`TotalShares = 1476`, `TVL = 3100`	`TotalShares = 1500`, `TVL = 3000`
7) Later, attacker executes a redeem for all his shares	`vToken.exchangeRateStored` already considers interests for block `56`	`vToken.exchangeRateStored` now considers interests for block `56`
8) Payout if attacker redeems at current accounting	Payout = `476 * 3100 / 1476 = 999`	Payout = `500 * 3100 / 1500 = 1033`
9) Attacker gain vs principal (`1000`)	Break-even	Profitable by `1033 - 1,000 = +33` units

Recommendation: Switch accrual idempotence guard to `block.number` and keep using timestamps for fee proration only.

Venus: Fixed in commit 9846031.

Cantina Managed: Fixed in the commit above. The `_accrueFees()` function is now using a new storage variable `_lastFeeAccrualBlock` to execute the accrual logic only once per block. Timestamps are still used for the fee proration.

4.2 Low Risk

4.2.1 Exit-fee HWM shield fails in drawdowns; performance fee skims retained exit-fee value from remaining LPs on recovery

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: The `Hub` charges an optional exit (redeem) fee that is retained in the vault rather than paid to any recipient: `previewWithdraw` grosses up the shares burned so that more share value is destroyed than underlying leaves, lifting price-per-share (PPS) for the remaining LPs (`Hub.sol:880-888`, `Hub.sol:901-912`). The stated design is that this fee is anti-sandwich friction that accrues to the remaining LPs, not protocol revenue (`Hub.sol:50-52`). Separately, the `Hub` charges a performance fee on genuine yield, measured as the gain of PPS above a monotonic high-water mark (HWM) in `_pendingPerformanceFee` (`Hub.sol:1462-1471`).

To stop the retained exit-fee uplift from later being re-counted as performance yield, `_withdraw` advances the HWM to the post-exit PPS after a fee-on redemption (`Hub.sol:919-929`). The advance is gated on an absolute new-high condition, `newPps > hwm` (`Hub.sol:924`):

```
if (_redeemFeeBps != 0) {
    uint256 ts = totalSupply();
    if (ts != 0) {
        uint256 newPps = (totalAssets() * EXP_SCALE) / ts;
        uint256 hwm = _highWaterMarkPerShare;
        if (newPps > hwm) {
```

```
            _highWaterMarkPerShare = newPps;
            emit HighWaterMarkUpdated(hwm, newPps);
        }
    }
}
```

When PPS sits below a previously recorded high (any drawdown), a fee-on exit lifts PPS but leaves `newPps < hwm`, so the branch is skipped and the retained exit-fee uplift is never recorded into the HWM. On a later recovery, `_pendingPerformanceFee` charges the performance fee on the whole gain above the stale HWM (`Hub.sol:1468-1470`), which now includes the exit-fee uplift that accrued while PPS was below the prior high. The root cause is the absolute `newPps > hwm` guard at `Hub.sol:924`: it only ratchets on new highs and never credits the retained-fee uplift during a drawdown.

Risk:

On any drawdown-then-recovery in PPS, the exit fee retained during the drawdown is taxed by the performance fee on recovery, and that slice is paid to `feeRecipient` instead of remaining with the non-exiting LPs, contradicting the retention design at `Hub.sol:50-52`. The misallocation is a value transfer from remaining LPs to `feeRecipient`, not a loss of principal. It is bounded to a second-order slice: the exit fee (capped at 5% via `MAX_REDEEM_FEE_BPS`) taxed at the performance rate (capped at 50% via `MAX_FEE_BPS`). It is reachable only once governance has enabled both a non-zero redeem fee and a non-zero performance fee; at the v1 `0/0/0` fee configuration the leak is dormant and no value is misallocated.

Recommendation: Consider making the exit-fee HWM shield incremental rather than absolute, so it also captures the retained-fee uplift while PPS is below a prior high. Two options:

1. Credit the retained exit-fee PPS uplift into `_highWaterMarkPerShare` unconditionally in `_withdraw`, dropping the `newPps > hwm` gate for the exit-fee component. This is storage-neutral and matches the stated intent at `Hub.sol:914-918`.
2. Maintain a separate cumulative exit-fee accumulator and subtract it from `profitAssets` in `_pendingPerformanceFee` (`Hub.sol:1469`), keeping the HWM for market gains only. This adds a storage field and must respect the upgrade-safe storage `__gap` invariant.

Venus: Fixed in commit 3bc80c7.

Cantina Managed: Fixed in the commit above.

The fix makes the exit-fee high-water-mark shield incremental instead of an absolute ratchet. `_withdraw` snapshots the pre-exit price-per-share `prePps` before the burn, and after the routed withdrawal credits the mark by the measured uplift `hwm + (newPps - prePps)` whenever the fee-on exit lifted the price-per-share, replacing the previous ratchet to the post-exit price-per-share under `newPps > hwm`.

```
+           uint16 redeemBps = _redeemFeeBps;
+           uint256 prePps;
+           if (redeemBps != 0) {
+               uint256 preTs = totalSupply();
+               if (preTs != 0) {
+                   prePps = (totalAssets() * EXP_SCALE) / preTs;
+               }
+           }
  //...
-           if (_redeemFeeBps != 0) {
+           if (redeemBps != 0) {
                uint256 ts = totalSupply();
                if (ts != 0) {
                    uint256 newPps = (totalAssets() * EXP_SCALE) / ts;
-                   uint256 hwm = _highWaterMarkPerShare;
-                   if (newPps > hwm) {
-                       _highWaterMarkPerShare = newPps;
-                       emit HighWaterMarkUpdated(hwm, newPps);
```

```
+                   if (newPps > prePps) {
+                       uint256 hwm = _highWaterMarkPerShare;
+                       uint256 newHwm = hwm + (newPps - prePps);
+                       _highWaterMarkPerShare = newHwm;
+                       emit HighWaterMarkUpdated(hwm, newHwm);
                    }
                }
            }
```

The mark is credited by exactly the retained-fee uplift whether the price-per-share sits above or below a prior high, so the drawdown-then-recovery path no longer taxes the retained fee. The delta is non-negative, so the mark stays monotonic, and genuine yield stays taxable since the mark moves by exactly the fee's contribution.

`_withdraw` is the only path that credits the exit-fee uplift and is the one changed.

4.2.2 Hub Share Pricing Can Use Stale FRV Lifecycle State

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `YieldGroupFRV` calls `updateVaultState()` before resource-level routing decisions, but `AdapterFRV.totalAssets()` does not update FRV state before reporting value to the Hub. As a result, Hub-level accounting can price shares using the FRV's last recorded lifecycle state rather than the state that would be reached if `updateVaultState()` were called first.

This can temporarily misprice Hub deposits, mints, withdrawals, redeems, and fee accrual. The most likely case is a stale `Fundraising` state after the vault is time-due to enter `Lock`, causing the Hub to value the FRV at principal only until a state-advancing call occurs.

Proof of Concept: Assume an FRV resource has:

```
Principal:       1,000 USDC
Accrued coupon:     72 USDC
Actual state if poked: Lock
Recorded state:  Fundraising
```

Because the recorded state is still `Fundraising`, `AdapterFRV.totalAssets()` reports only principal:

```
Reported FRV value: 1,000 USDC
Actual marked value after state update: 1,072 USDC
```

Recommendation: Consider ensuring the FRV lifecycle state is current before Hub-level pricing paths rely on FRV `totalAssets()`. One possible mitigation is to call `updateVaultState()` from the FRV valuation path where safe and acceptable.

Venus: Fixed in commit 922d3fa.

Cantina Managed: Fixed by adding an `accrue()` on the FRV level.

4.2.3 Emergency Reallocation Cannot Exit Locked FRV Positions

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `Hub.emergencyReallocate()` can be called while the Hub is paused, but it still depends on the underlying YieldGroup and resource withdrawal rules. FRV vaults are only withdrawable in terminal states such as `Matured`, `Failed`, or `Liquidated`. During `Fundraising`, `Lock`, `Pending Settlement`, or `SettlementDeadlineExceeded`, FRV `maxWithdraw()` is zero and withdrawal attempts cannot extract the locked value.

This means emergency reallocation cannot be relied on to move capital out of an active locked FRV position. It can route around FRV exposure or move other liquid assets, but it cannot unlock FRV principal before the FRV vault itself becomes withdrawable.

Proof of Concept: Assume the Hub has:

```
Core liquid assets: 300,000 USDC
Locked FRV assets: 700,000 USDC
```

An FRV-related risk event occurs while the FRV is still in `Lock`. Governance pauses the Hub and calls `emergencyReallocate()` to pull funds from the FRV YieldGroup into Core.

The FRV resource reports no withdrawable liquidity:

```
AdapterFRV.maxWithdraw(resource, YieldGroupFRV) = 0
```

Any attempt to withdraw from the locked FRV fails because the FRV vault is not in a terminal state. Emergency reallocation cannot move the `700,000 USDC` locked in FRV. User withdrawals remain limited to the liquid Core assets and any other withdrawable resources.

Recommendation: Treat emergency reallocation as a routing and containment tool, not as an FRV unlock mechanism. In the current design, it is not possible to move any funds from a locked FRV even with the emergency flow.

Venus: Fixed in commit 91231d1.

Cantina Managed: Fixed in the commit above by clarifying that aspect in the documentation.

4.2.4 Redeemers can front-run an FRV settlement to exit at the pre-loss mark, socializing the shortfall to remaining holders

Severity: Low Risk.

Context: AdapterFRV.sol#L248-L269.

Description: The FRV over-mark that inflates `Hub.totalAssets()` during the `Lock`, `Pending Settlement`, and `SettlementDeadlineExceeded` states, and the resulting advantage to earlier redeemers, is documented in the finding "FRV Accrued Yield Can Inflate Hub NAV Before It Is Liquid Or Certain". This finding isolates the atomic front-running vector on top of it.

The mark steps down to the real value only when the loss-realization transaction lands, a short settlement or a settler liquidation, and that transaction is permissionless and observable in the public BSC mempool. A redeemer who sees it pending submits a Hub `redeem` at higher priority in the same block and captures the pre-loss mark: `previewRedeem` prices the payout from `totalAssets()` before `_routeWithdraw`, while the `updateVaultState()` step-down only happens later in the cascade at `YieldGroupBase.sol:554`, so the payout is locked at the stale mark and paid from the Hub's other liquid resources while the FRV reports `maxWithdraw == 0` (`AdapterFRV.sol:141-143`).

Risk:

A redeemer who front-runs the loss-realization transaction exits at the full pre-loss price-per-share, paid from the Hub's other liquid resources, while the remaining liquidity providers absorb the realized shortfall. The extraction is atomic and permissionless, needs no privileged role, and is bounded by the FRV YieldGroup's fraction of TVL; at the v1 `0/0/0` configuration no exit-fee friction opposes it.

Recommendation: Consider having the settler submit the loss-realization transaction through a private relay such as bloXroute on BSC, so it is not visible in the public mempool and cannot be front-run. This is an off-chain measure by the external settler, does not bind the protocol, and is weaker on BSC than on chains with mature private-transaction infrastructure. It does not address a liquidity provider that redeems at the optimistic mark without front-running a specific transaction, which is the broader window covered by finding #2.

Venus: Acknowledged, this is the atomic front-run variant of the finding "FRV Accrued Yield Can Inflate Hub NAV Before It Is Liquid Or Certain", same root cause and disposition. The private relay suggestion is a fair off-chain operator control but doesn't bind the protocol. Full context is in the PR 2 thread linked on the aforementioned.

Cantina Managed: Acknowledged. The accepted residual is a redeemer front-running the FRV loss-realization transaction to exit at the pre-loss mark and leave the shortfall to the remaining holders, bounded by the FRV cap on Hub NAV, the low likelihood of a shortfall from vetted counterparties, and the treasury backstop. The private relay is an off-chain operator control that does not bind the protocol.

4.2.5 Selector collision can convert skippable external protocol reverts into hard withdraw failures

Severity: Low Risk.

Context: YieldGroupBase.sol#L561-L568, YieldGroupBase.sol#L650-L673.

Description: The role of the `YieldGroupBase._rethrowIfAdapterUnderfilled()` function is to use `bytes4` selector equality to classify potential revert data caught when dispatching a withdrawal via an adapter:

- If it matches the selector of the `AdapterUnderfilled` error (which is `0x273121dd`), the function halts the whole withdrawal loudly with a revert.
- If not, any other revert (external-protocol pause, cap, reentrancy-blocked leg) is left for the parent function `_tryWithdrawOneResource()` to skip.

Because selectors are only encoded using 4 bytes, there can have `2^32 = 4,294,967,296` possible values, and collisions can happen. In other words, it is possible for errors returned by the integrated protocol to share the same colliding selector. In that case, it would turn a recoverable paused/cap/revert condition into an atomic withdraw failure. This is a one-way DoS-style behavior and does not create direct fund theft. Still, it could negatively disrupt a withdrawal route.

Recommendation: Consider removing the collision surface by returning a boolean in the `_dispatchWithdraw()` function that will be `true` if the exact expected amount was received, instead of reverting with `AdapterUnderfilled`. Use the value of that boolean in parent functions.

Venus: Fixed in commit 73f18ec.

Cantina Managed: Fixed in the commit above. The collision surface was removed. The `_rethrowIfAdapterUnderfilled()` function was removed. The `_dispatchWithdraw()` function now returns the exact amount received, and the parents functions `withdrawResource()` and `_tryWithdrawOneResource()` use that value to conditionally revert with `AdapterUnderfilled`.

4.2.6 Urgent reallocation blocked when all destination caps are fully utilized

Severity: Low Risk.

Context: Hub.sol#L1151-L1178.

Description: When a reallocation is executed, each deposit leg must pass destination capacity checks before funds can be moved. Both `reallocate()` and `emergencyReallocate()` require exact, balanced execution (total withdrawn == total deposited) and revert if any leg is underfilled.

Each push leg must satisfy destination `YieldGroup` capacity. The reallocation reverts when `currentBal + amount > effectiveCap` for that destination, where `cap` is derived from the `YieldGroup`'s configured cap settings and the reallocation TVL snapshot.

If an incident affects one leg and all destination YieldGroups/resources are already at full capacity, funds cannot be migrated out of the risky source leg because there is no destination room.

This can only be solved by increasing destination caps (`YieldGroup` cap via `raiseYieldGroupCap()`, and resource cap via `raiseResourceCap()`). But those are VIP-only actions. Operator-only roles can only tighten caps.

As VIP execution is timelocked, response is delayed, so this creates a real incident-reaction latency even though emergencyReallocate exists.

Proof of Concept:

- `YG_A` : cap `100` , balance `100` ;
- `YG_B` : cap `100` , balance `90` (incident/at-risk source);
- Urgent need to migrate `90` from `YG_B` to `YG_A` ;
- During reallocation attempt, withdrawing from `YG_B` works, but deposit to `YG_A` reverts because `100 + 90 > 100` ;
- Rebalance is blocked until a VIP raises destination capacity of `YG_A` .

Recommendation: Consider any of these options:

1. A governance fast-track path for urgent cap raises;
2. A pre-provisioned emergency sink destination with persistent spare capacity;
3. A no-yield uncapped emergency sink destination;

Venus: Acknowledged. The current design already resolves the scenario in a single timelocked VIP: `raiseYieldGroupCap` (or `raiseResourceCap`) followed by `emergencyReallocate` , with the cap read from fresh storage at push time so it binds in the same execution. `emergencyReallocate` is itself a VIP action; its ”emergency” property is bypassing the Hub pause, not the timelock.

To shorten incident response below the governance timelock (your option 1), we will grant a fast-acting guardian role the relevant ACM permissions: `raiseYieldGroupCap` / `raiseResourceCap` and `emergencyReallocate` . This needs no contract change.

Cantina Managed: Acknowledged.

4.2.7 Collateral underdelivery compensation from Failed FRVs is outside Liquidity Hub token accounting and payout flow

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: The documentation of FRVs (https://docs-v4.venus.io/technical-reference/reference-technical-articles/fixed-rate-vaults#state-machine) describes the lifecycle of a vault. It starts with the borrower that needs to deposit a margin, in collateral assets, to move to the `Fundraising` state. Then, during the `Fundraising` state:

1. The suppliers are expected to deposit what will be borrowed (in hub assets);
2. The borrower is expected to deposit the rest of the collateral (in collateral assets);

If both requirements are met, the FRV moves to the `Lock` state. However, if the second requirement is not met, the FRV moves to the `Failed` state.

In that case of collateral underdelivery (second requirement not met), the initial margin (in collateral assets) deposited by the borrower will be confiscated. Each withdraw / redeem from the suppliers triggers a call to `_afterWithdrawHook()`, which distributes a pro-rata slice of the remaining confiscated margin (in collateral assets).

The Liquidity Hub does not mention that edge case of collateral underdelivery in its documentation, and there is no mechanism in the `YieldGroupBase` contract to handle other assets than `hub.asset()`, and to fairly distribute these compensation assets to suppliers.

Recommendation: Consider assessing if the Liquidity Hub should handle that case.

Venus: Agree, on a Scenario-B Failed FRV the vault pays the confiscated margin in the collateral token to the Hub, and since Hub accounting is single-asset it sits as an unaccounted balance recoverable only via governance sweep.

We are addressing it operationally rather than in-contract (no oracle/converter in the withdraw path). On a default, governance monitors MarginConfiscated / MarginCompensationClaimed, sweeps the collateral off the Hub, liquidates it off-chain, and donates the proceeds back to the Hub so the recovery lifts share price for holders.

Also, added documentation in commit 909afe22.

Cantina Managed: Acknowledged. Scenario handled operationally, and now documented in the README.

4.3 Informational

4.3.1 Resource Accrual Failures Are Silently Skipped

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `YieldGroup.accrue()` attempts to accrue every registered resource, but silently ignores any adapter-level accrual failure:

```
try IResourceAdapter(_resourcesConfig[list[i]].adapter).accrue(list[i]) { } catch { }
```

This is intentionally fail-open so one broken or paused resource does not block Hub-level fee accrual or user flows. However, because the catch block emits no event, operators and off-chain monitors have no on-chain signal that a resource failed to accrue.

Recommendation: Emit a low-cost event in the catch block with the resource and adapter address.

For example:

```
event ResourceAccrualFailed(address indexed resource, address indexed adapter);
```

Venus: Fixed in commit 8e1b305.

Cantina Managed: Fixed in the commit above by emitting an event in the catch block.

4.3.2 FRV Marked Value Can Consume YieldGroup Cap Room

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Hub reallocation enforces destination YieldGroup caps against the YieldGroup's current `totalAssets()`:

```
uint256 cap = _effectiveCap(cfg, taSnapshot);
uint256 currentBal = IYieldGroupBase(depositLeg.yieldGroup).totalAssets();

if (currentBal + depositLeg.amount > cap) {
    revert HubCapacityExceeded(depositLeg.amount, cap > currentBal ? cap - currentBal :
    ↪  0);
}
```

For FRV YieldGroups, `totalAssets()` can increase over time because the adapter marks accrued fixed-rate value. This can cause an FRV YieldGroup to drift up to, or above, its configured cap without any new deposit.

Once that happens, future reallocation pushes into the FRV YieldGroup can revert even if the operator is trying to deposit into a different FRV resource under the same YieldGroup.

Proof of Concept: An FRV YieldGroup has an effective cap of `800,000 USDC`.

The operator reallocates:

```
FRV principal: 780,000 USDC
Cap room:       20,000 USDC
```

Over time, FRV accrued value increases the YieldGroup's reported NAV:

```
FRV totalAssets(): 835,000 USDC
Effective cap:     800,000 USDC
```

The YieldGroup is now above cap due to accrued marked value. Existing exposure is not force-unwound, but any later push into that YieldGroup fails:

```
currentBal + depositLeg.amount > cap
```

This can block reallocation into FRV even when the target resource itself has deposit capacity.

Recommendation: During re-allocation, be aware that re-allocating large portions of the funds to FRV vaults can cause the total amount held plus the fixed rate interest to be more than the cap, effectively blocking more deposits. Add relevant documentation to guard against this case.

Venus: Fixed in commit 6d9e91b.

Cantina Managed: Fixed in the commit above by adding the relevant docs.

4.3.3 Consolidated suggestions to clarify the documentation

Severity: Informational.

Context: README.md#L210-L221.

Description: When the review started, the content of the `README.md` was partially outdated. For instance, it described a Hub-level "APY tracking" feature that was not implemented. During the review, the Venus team clarified the intention.

In addition, there is a mismatch between the NatSpec of the `_tryWithdrawOneResource()` function:

```
//     *      YieldGroup's receipt tokens and credits underlying back to this YieldGroup.
//     *      Defensively verifies that the YieldGroup's balance dropped by exactly
 ↪  `toDeliver`
//     *      after the adapter's internal `safeTransfer` to `to`.
```

And the recipient, as well as the balance check performed by the `_dispatchWithdraw()` function, which checks the address `to` instead of the address of the YieldGroup:

```
function _dispatchWithdraw(address adapter, address resource, uint256 amount, address
↪  to) private {
    IERC20 assetToken = IERC20(_assetToken);
    uint256 preBalTo = assetToken.balanceOf(to);
    adapter.functionDelegateCall(abi.encodeCall(IResourceAdapter.withdraw, (resource,
    ↪  amount, to)));
    uint256 received = assetToken.balanceOf(to) - preBalTo;
    if (received != amount) revert AdapterUnderfilled(resource, amount, received);

    emit WithdrawRouted(resource, amount);
}
```

Recommendation: Consider:

1. Updating the content of the `README.md` file to match the current implementation.
2. Aligning the Natspec of the `_tryWithdrawOneResource()` function with the code of the `_dispatchWithdraw()` function.

Venus: Fixed in commits 5b6c9f2 and 73f18ec.

Cantina Managed: Fixed in the commits above. Both items were clarified.

4.3.4 A same-block deposit after unpause skips Core interest accrual, pricing shares on a stale rate and skimming pending interest

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Before it prices an ERC4626 deposit, the `Hub` refreshes the Core market's exchange rate inside `_accrueFees`, which runs at the start of every deposit. The refresh, `_accrueResources`, pokes each Core market's `accrueInterest` so `totalAssets` reads a current rate. `_accrueFees` has a once-per-block guard that early-returns before that refresh:

```
// _accrueFees (Hub.sol:990-1002)
uint64 nowTs = uint64(block.timestamp);
uint64 last = _lastFeeAccrualTimestamp;
if (nowTs == last) return;   // returns BEFORE _accrueResources()

_accrueResources();          // pokes vToken.accrueInterest() -> fresh exchange rate
```

`unpauseHub` advances that timestamp cursor to the current block without poking resources:

```
// unpauseHub (Hub.sol:386)
_lastFeeAccrualTimestamp += uint64(block.timestamp) - _pauseStart;
```

An attacker can line the cursor up so the first deposit after unpause skips the refresh:

1. Front-run the pending `pauseHub()` with permissionless `accrueFees()`, so `_lastFeeAccrualTimestamp` equals the pause block, which `pauseHub` also stores as `_pauseStart`.
2. When governance later calls `unpauseHub()`, the advance lands `_lastFeeAccrualTimestamp == block.timestamp`.
3. Back-run the unpause in the same block with `deposit()`. `_accrueFees` sees `nowTs == last`, early-returns, and skips `_accrueResources`.

Pricing then uses the stale `exchangeRateStored` in `totalAssets`:

```
// AdapterCoreV1.totalAssets (AdapterCoreV1.sol:176)
uint256 gross = (vBal * IVToken(resource).exchangeRateStored()) / EXP_SCALE;
```

With the stale rate the NAV is understated, so the deposit mints the attacker extra shares. Routing the deposit then calls `vToken.mint`, which refreshes the rate after the price was already set, so those extra shares claim a slice of the pending interest.

Risk:

No principal is at risk. The attacker only skims a tiny sliver of Core interest that was not yet booked, on the order of cents, taken from existing depositors, and only when governance pauses and then unpauses the Hub while the attacker front-runs the pause and back-runs the unpause, so the opportunity is rare.

Recommendation: Consider stepping the fee cursor back by one in `unpauseHub` when it lands on the current block, so the next `_accrueFees` refreshes the resources instead of skipping them:

```
_lastFeeAccrualTimestamp += uint64(block.timestamp) - _pauseStart;
if (_lastFeeAccrualTimestamp == uint64(block.timestamp)) {
    _lastFeeAccrualTimestamp = uint64(block.timestamp) - 1;
}
```

This is contained to `unpauseHub` and only costs one extra second of management-fee proration.

Venus: Fixed in commit 6fdfcd4.

Cantina Managed: Fixed in the commit above. `unpauseHub` now steps the fee cursor back by one when the pause-duration advance would land it on the unpause block, so the first deposit in that block runs `_accrueResources` and prices against a refreshed Core rate instead of the stale one. The only side effect is one extra second of management-fee proration, which is negligible.

> ℹ️ **Note:**
> The later commit 9846031 replacing the timestamp-based guard with a block-based made that fix no longer required, so it was reverted.